

Framework For Team

MSME

INSTALL GITHUB COPILOT!

Components

Platform	What It Does Well	What It Lacks (Your Niche)
IBM Quantum Composer / Qiskit	Excellent drag-and-drop circuit builder and state visualization.	No conversational AI prompt agent translating natural language into live circuits automatically.
Quirk (by Craig Gidney)	Instant drag-and-drop quantum state visualizer.	Static UI; requires manual gate placement with no AI explanation layer.
Generative AI Chatbots (ChatGPT / Claude)	Can write Qiskit code from natural language prompts.	Lack integrated real-time drag-and-drop visual circuit canvases, live statevector graphs, or automated error diagnosis loops.
Quantum Computing Studio / Educational Apps	AI chatbots paired with learning notes.	Very few feature a local agentic framework capable of parsing broken circuit states and auto-correcting them on a canvas in real time.

Divisions: MVP

Features:

- AI Snippet per operation of description
- english to code translation -> then composer translation all in one
- Codechef's: Learn by Doing Philosophy
- A

Frontend Layer

- **React:** Core UI rendering framework.
- **React Flow / @xyflow/react** : Interactive visual workspace for constructing circuits via drag-and-drop nodes and custom gate handles.
- **Recharts / Chart.js:** Real-time statevector visualizer for drawing qubit measurement probability distributions.
- **Three.js / @react-three/fiber** : 3D rendering engine to plot and animate quantum state transformations on a dynamic Bloch sphere.

Backend & Agent Layer

- **Python 3.11+ & FastAPI:** High-performance REST/WebSocket framework for asynchronous communication between the client and LLM agent.
- **Hermes Agent Framework (Local LLM):** Locally hosted LLM agent responsible for converting prompt instructions into valid Qiskit code.
- **LangChain / LlamaIndex:** Framework to manage agent memory, structured JSON output formats, and prompt templates.
- **Docker & Sandbox (`exec` / `Subprocess`):** Isolated container execution environment to validate generated Python snippets safely before processing.

Quantum Computing & Simulation

- **Qiskit (`qiskit-terra`):** Primary SDK to construct, manipulate, and analyze quantum circuits programmatically.
- **Qiskit Aer (`qiskit-aer`):** High-performance C++ backend simulator for calculating statevectors, unitary matrices, and probability counts on classical CPUs/GPUs. [GitHub Pages](#)
- **Cirq & `qsim` (Alternative Engine):** Lightweight fallback for rapid Google-style circuit operations.

DevOps & Infrastructure

- **Zsh / Bash Scripting:** Automated deployment and lifecycle management scripts for local AI model orchestration.
- **Ollama / vLLM:** Local inference engine to serve LLM models with minimal response latency.

What Makes Your Proposal Unique (Your Competitive Moat)

1. **Natural Language to Real-Time Circuit Compilation:** Users type intent in plain English, and the platform programmatically compiles and renders the circuit visually rather than just dropping Qiskit code blocks in a chat box.
2. **Interactive Debugging Loop:** Modifying a circuit manual-first and having an AI explain *why* state breakdown occurred (e.g., losing entanglement) bridges classical intuition with quantum behavior.
3. **Deterministic Verification Layer:** Leveraging a local backend model (like Hermes) solely for intent extraction while running classical simulations via Qiskit/Cirq guarantees zero mathematical hallucination.